

ÉCOLE NATIONALE SUPÉRIEURE DE
L'ÉLECTRONIQUE ET DE SES APPLICATIONS

RAPPORT DE PROJET — SEMESTRE 6

OSCILLOSCOPE PORTABLE CONNECTÉ

2025-2026

Étudiants :

JABOU Yassine · ACHBAD Younes
BOUDOUX d'HAUTEFEUILLE Gabriel · OZDEN Gökay

Remerciements

Nous remercions nos encadrants de l'ENSEA pour leur accompagnement tout au long du semestre, et particulièrement pour leur compréhension lors des difficultés d'approvisionnement que nous avons rencontrées. Leurs retours lors des séances de suivi ont été déterminants pour prioriser nos efforts sur ce qui était réalisable dans les délais.

Merci également au laboratoire de l'école pour l'accès au matériel de mesure et aux postes de soudure du FabLab

Résumé

Ce rapport présente la conception et le développement d'un oscilloscope portable connecté, réalisé dans le cadre du projet de semestre 6 à l'ENSEA. Le projet s'articule autour d'un boîtier d'acquisition matériel (PCB sur STM32L476RG) et d'une application Android Java pour la visualisation des signaux via Bluetooth.

Note préliminaire sur l'état d'avancement : un retard critique de livraison de composants chez notre fournisseur principal (Mouser) a empêché l'assemblage final du PCB dans les délais du projet. Cette contrainte, indépendante de notre volonté, n'a pas remis en cause la conception — le routage KiCad est 100 % finalisé et validé (DRC OK) — mais elle a conduit à orienter la validation vers une approche simulation logicielle. L'ensemble du système est « ready-to-plug » : dès réception et soudure des composants manquants, la chaîne complète peut être mise en service sans modification de conception.

Ce qui a été pleinement développé et validé :

- PCB double couche $\sim 24 \times 95$ mm : schéma complet, routage terminé, DRC validé, carte reçue du fabricant
- Firmware STM32 en C (HAL) : acquisition ADC 12 bits, DMA circulaire double buffer à 10 kHz, driver PGA849, protocole Bluetooth 0xAA/0x55, machine à états
- Application Android Java : affichage temps réel, trigger réglable, FFT 512 points (fenêtre de Hann), curseurs $\Delta T/\Delta V$, mesures automatiques, mode simulation complet
- Validation protocole complète : liaison UART/Bluetooth testée, endianness corrigé, trames vérifiées

Abstract

This report presents the design and development of a portable connected oscilloscope, completed as part of the semester 6 project at ENSEA. The system consists of a hardware acquisition board (PCB around STM32L476RG) and a Java Android application for wireless signal visualization over Bluetooth.

A critical component delivery delay prevented final PCB assembly within the project timeline. The KiCad routing is fully complete and DRC-validated; the system is ready-to-plug pending soldering. All software components — STM32 firmware (C, HAL, DMA), Bluetooth protocol, and the Android application (real-time display, FFT, trigger, cursors, simulation mode) — have been fully developed and validated through software simulation and partial hardware testing.

1. Introduction

1.1 Contexte et motivation

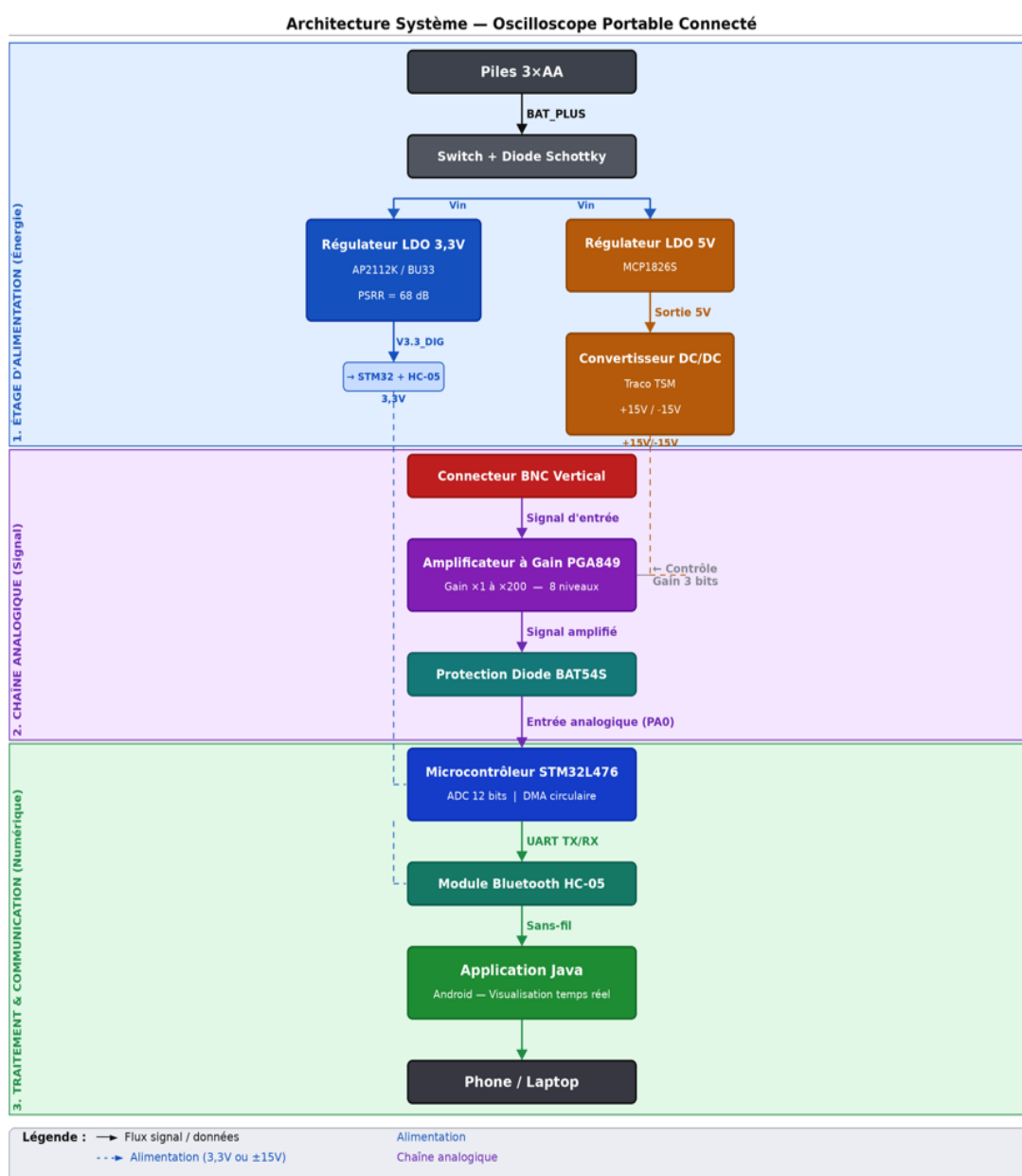
L'oscilloscope est l'instrument de base de tout laboratoire d'électronique. Il permet de visualiser en temps réel l'évolution d'un signal, de mesurer son amplitude et sa fréquence, et de diagnostiquer le comportement d'un circuit. Les oscilloscopes de banc professionnels sont efficaces mais encombrants et coûteux — souvent hors de portée pour un usage nomade ou terrain.

Ce projet visait à concevoir une alternative compacte : un oscilloscope embarqué alimenté sur batterie, qui transmet les données à un smartphone Android via Bluetooth. Toute la puissance d'affichage et de traitement du signal est déportée sur l'application mobile, ce qui simplifie le firmware et permet une interface bien plus riche qu'un écran embarqué de quelques centimètres.

1.2 Cahier des charges

Les spécifications fixées en début de projet étaient les suivantes :

- 2 voies d'entrée analogiques (1 voie implémentée dans la V1)
- Bande passante : 10 Hz – 50 kHz
- 1024 points par capture en mode dual, 2048 en mode simple
- Transmission Bluetooth sans dégradation vers application Android
- Autonomie visée : 6 heures sur batterie
- Interface de visualisation complète sur smartphone



- Fig. 1 — Architecture globale du système : BNC → PGA849 → STM32 → HC-05 → Application Android Java.

1.3 Déroulement réel du projet et état final

Le projet a été conduit sur environ quatre mois, entre les contraintes académiques habituelles (cours, partiels) et les aléas propres à tout projet hardware : délais fournisseurs, itérations sur le schéma, débogage firmware. La chronologie réelle s'est articulée autour de trois phases :

1. Phase de conception (semaines 1–5) : spécifications, choix des composants, schéma KiCad, création des empreintes personnalisées (PGA849 QFN-16, TSM0515D).
2. Phase de développement logiciel (semaines 4–10) : firmware STM32 en parallèle du PCB, développement de l'application Android Java avec mode simulation.
3. Phase d'intégration (semaines 11–14) : routage PCB finalisé, DRC validé, carte envoyée en fabrication. Validation firmware + Bluetooth sur Nucleo. Assemblage PCB non finalisé (retard composants Mouser).

2. Présentation Générale du Projet

2.1 Architecture système

Le système se découpe en deux sous-ensembles communiquant sans fil :

- Le boîtier embarqué : chaîne analogique (BNC → PGA849 → ADC) + STM32 (firmware C, DMA, Bluetooth UART) + alimentation batterie.
- L'application Android Java : réception Bluetooth, décodage protocole, affichage temps réel, traitement signal (FFT, trigger, curseurs), simulation intégrée.

L'idée de déporter l'affichage sur le smartphone a été validée dès le départ. Cela évite d'intégrer un écran dans le boîtier, réduit la consommation, simplifie le firmware, et offre une interface incomparablement plus riche qu'un petit OLED.

2.2 Synthèse des choix techniques

Composant / Choix	Justification technique
STM32L476RG (NUCLEO)	ADC 12 bits intégré, famille L4 basse consommation, HAL documentée, familier des TPs
PGA849 (Texas Instruments)	Gain programmable 3 bits ($\times 1$ à $\times 200$), faible bruit, piloté par 3 GPIO STM32
BU33SD5WG (ROHM, SSOP-5)	LDO 3,3 V, PSRR = 68 dB — filtre le ripple HF du Traco pour protéger l'ADC
TSM0515D (Traco Power)	Convertisseur DC/DC ± 15 V isolé depuis 5 V pour alimentation symétrique PGA849
BAT54S (Schottky)	Double protection : inversion polarité batterie + clamping surtension entrée ADC
HC-05 Bluetooth SPP	UART transparent, profil SPP, simple à interfacer STM32 et Android
DMA Circulaire double buffer	Acquisition déterministe 10 kHz, zéro perte, charge CPU ≈ 0 % pendant l'ADC

Application Android Java

Interface riche (FFT, trigger, curseurs), mode simulation intégré, pas d'écran embarqué

3. Partie Hardware

3.1 État d'avancement — rappel de contexte

Les sections qui suivent décrivent la conception telle qu'elle a été réalisée. Les justifications techniques sont celles qui ont guidé les choix. Les résultats expérimentaux présentés en section 6 correspondent uniquement à ce qui a effectivement pu être testé.

3.2 Alimentation — Conception de la chaîne d'énergie

3.2.1 Contraintes et architecture

La chaîne d'alimentation est le sous-système qui a demandé le plus de réflexion, pour deux raisons :

- Le PGA849 nécessite une alimentation symétrique ($\pm V_S$) pour amplifier des signaux centrés autour de 0 V sans saturation.
- L'ADC 12 bits de la STM32 a une résolution de $1 \text{ LSB} = 3300/4096 \approx 0,806 \text{ mV}$. La moindre perturbation haute fréquence sur la tension d'alimentation se retrouve directement dans les échantillons numérisés.

Ces deux contraintes ont conduit à une architecture cascade à deux branches, illustrée ci-dessous :

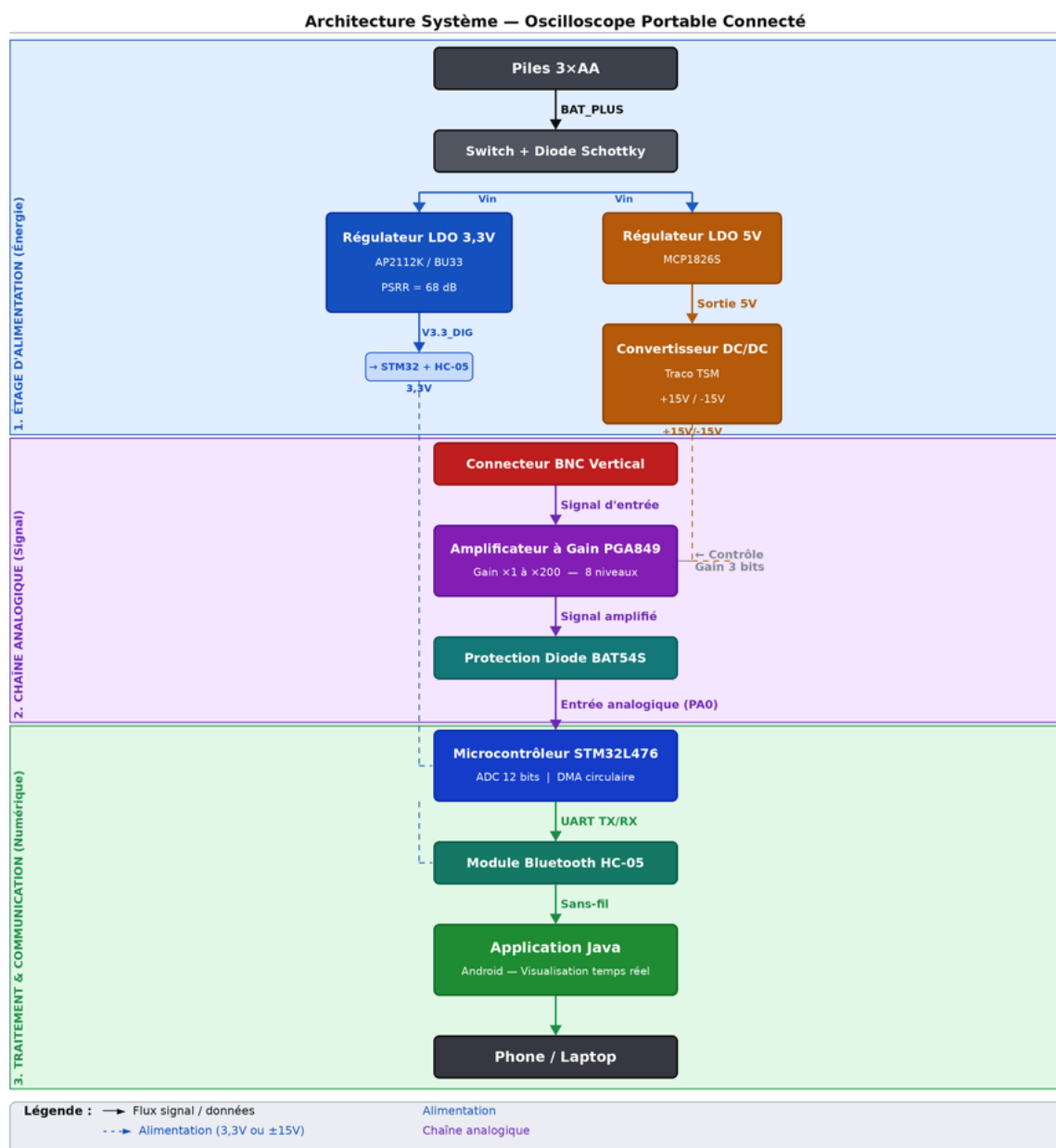


Fig. 1 — Architecture globale du système : BNC → PGA849 → STM32 → HC-05 → Application Android Java.

3.2.2 Branche 1 — Alimentation ± 15 V pour le PGA849

Les piles 3×AA (4,5 V nominal) alimentent d'abord un LDO MCP1826S qui fournit un 5 V stabilisé. Ce 5 V est ensuite converti en ± 15 V symétrique par le TSM0515D de Traco Power, un convertisseur DC/DC isolé à découpage.

Le TSM0515D est un module tout-intégré qui simplifie considérablement la conception : pas besoin de transformer, de contrôleur PWM, ni de bobine externe. Il génère cependant un ripple HF caractéristique des convertisseurs à découpage — ce qui impose le soin apporté à la branche 3,3 V.

3.2.3 Branche 2 — 3,3 V propre pour le STM32 : le rôle clé du BU33SD5WG

La branche 3,3 V est critique. Elle alimente directement le STM32 (donc son ADC) et le module Bluetooth. Pour cette branche, nous avons sélectionné le LDO BU33SD5WG de ROHM (boîtier SSOP-5, 500 mA max) à la place d'un LDO générique.

Le critère déterminant est son PSRR (Power Supply Rejection Ratio) exceptionnel de 68 dB. Ce paramètre mesure la capacité du régulateur à atténuer les perturbations présentes sur sa tension d'entrée avant qu'elles n'atteignent la sortie. En termes concrets :

PSRR = 68 dB → atténuation = $10^{(68/20)} \approx 2512\times$

Autrement dit, un bruit de 100 mV crête sur l'entrée du BU33SD5WG (généré par le Traco) n'apparaît qu'à $\sim 40 \mu\text{V}$ sur la tension 3,3 V fournie à l'ADC. Or à 12 bits, 1 LSB = 0,806 mV : cette perturbation résiduelle de 40 μV représente moins de 0,05 LSB, ce qui est parfaitement négligeable. Un LDO standard (PSRR typique 40 dB) aurait laissé passer $\sim 1 \text{ mV}$, soit plus d'un LSB de bruit parasite.

Ce choix est donc la clé de la précision de mesure du système, et pas un détail de conception.

3.2.4 Protection en entrée — Diode Schottky et BAT54S

Deux protections sont intégrées en entrée du circuit :

- Diode Schottky série : protège l'ensemble du circuit contre une inversion de polarité lors de l'insertion des piles. Le seuil bas ($\sim 0,3 \text{ V}$) minimise la chute de tension sur la branche d'alimentation.
- BAT54S (double Schottky en boîtier SOT-23) : utilisée comme diode de clamping sur l'entrée analogique. En cas de signal d'entrée dépassant les rails d'alimentation du PGA, les diodes Schottky écrètent la tension avant qu'elle n'atteigne l'ADC. Les diodes Schottky sont privilégiées ici pour leur faible capacité parasite, qui n'altère pas la bande passante du signal.

Composant	Référence	Rôle	Valeur clé
LDO 5 V	MCP1826S	5 V stabilisé pour le Traco	1 A max
DC/DC $\pm 15 \text{ V}$	TSM0515D (Traco)	Alimentation symétrique PGA849	Isolé, $\pm 15 \text{ V}$ / 70 mA
LDO 3,3 V	BU33SD5WG (ROHM)	3,3 V propre STM32 + HC-05	PSRR = 68 dB
Découplage HF	100 nF $\times$ 10 (CMS)	Filtrage hautes fréquences	Placés au plus près des ICs
Filtrage BF	10 μF $\times$ 3 (électro)	Stabilisation basse fréquence	Sorties régulateurs
Protection polarité	Schottky série	Inversion polarité batterie	$V_f \approx 0,3 \text{ V}$
Protection ADC	BAT54S	Clamping surtension entrée	Faible capacité parasite

3.2.5 Estimation d'autonomie

Avec 3 piles AA de 2500 mAh et une consommation totale estimée à $\sim 400 \text{ mA}$ (STM32 $\sim 30 \text{ mA}$, PGA849 $\sim 10 \text{ mA}$, HC-05 $\sim 40 \text{ mA}$, TSM0515D $\sim 100 \text{ mA}$ équivalent avec pertes) :

Autonomie estimée $\approx 2500 \text{ mAh} / 400 \text{ mA} \approx 6,25 \text{ heures}$

Cette valeur est cohérente avec l'objectif du cahier des charges (6 heures). Elle n'a pas pu être vérifiée expérimentalement faute d'avoir assemblé le circuit complet.

3.3 Chaîne d'acquisition analogique

3.3.1 Amplificateur à gain programmable PGA849

Le PGA849 de Texas Instruments est l'élément central de la chaîne d'acquisition. Il amplifie le signal d'entrée avec un gain sélectionnable sur 8 niveaux, codés en binaire sur 3 broches logiques (A0, A1, A2) pilotées par les GPIO PA4, PA5, PA6 de la STM32.

Le signal d'entrée arrive via un connecteur BNC vertical (TE Connectivity 1478204), standard en laboratoire. La sortie du PGA attaque directement l'entrée analogique de l'ADC1 (broche PA0, canal IN5 de la STM32).

Code A2:A1:A0	Gain	Plage d'entrée typique
000	×1	> 1 V crête — signaux forts
001	×2	500 mV – 1 V
010	×5	200 – 500 mV
011	×10	100 – 200 mV
100	×20	50 – 100 mV
101	×50	20 – 50 mV
110	×100	10 – 20 mV
111	×200	< 10 mV — signaux très faibles

3.3.2 Résolution de l'ADC

L'ADC1 de la STM32L476 est configuré en 12 bits, référence $V_{ref} = 3,3 \text{ V}$:

1 LSB = 3300 mV / 4096 $\approx$ 0,806 mV (gain ×1)

Au gain maximum ×200 du PGA, la résolution effective sur le signal d'entrée descend à 0,806 / 200 $\approx$ 4 $\mu\text{V/bit}$ en théorie. En pratique, le bruit de fond du PGA849 et du câble d'entrée fixerait la résolution réelle à quelques dizaines de μV — ce qui reste excellent pour la plage de fréquences visée.

3.4 PCB — Conception et routage

3.4.1 Caractéristiques

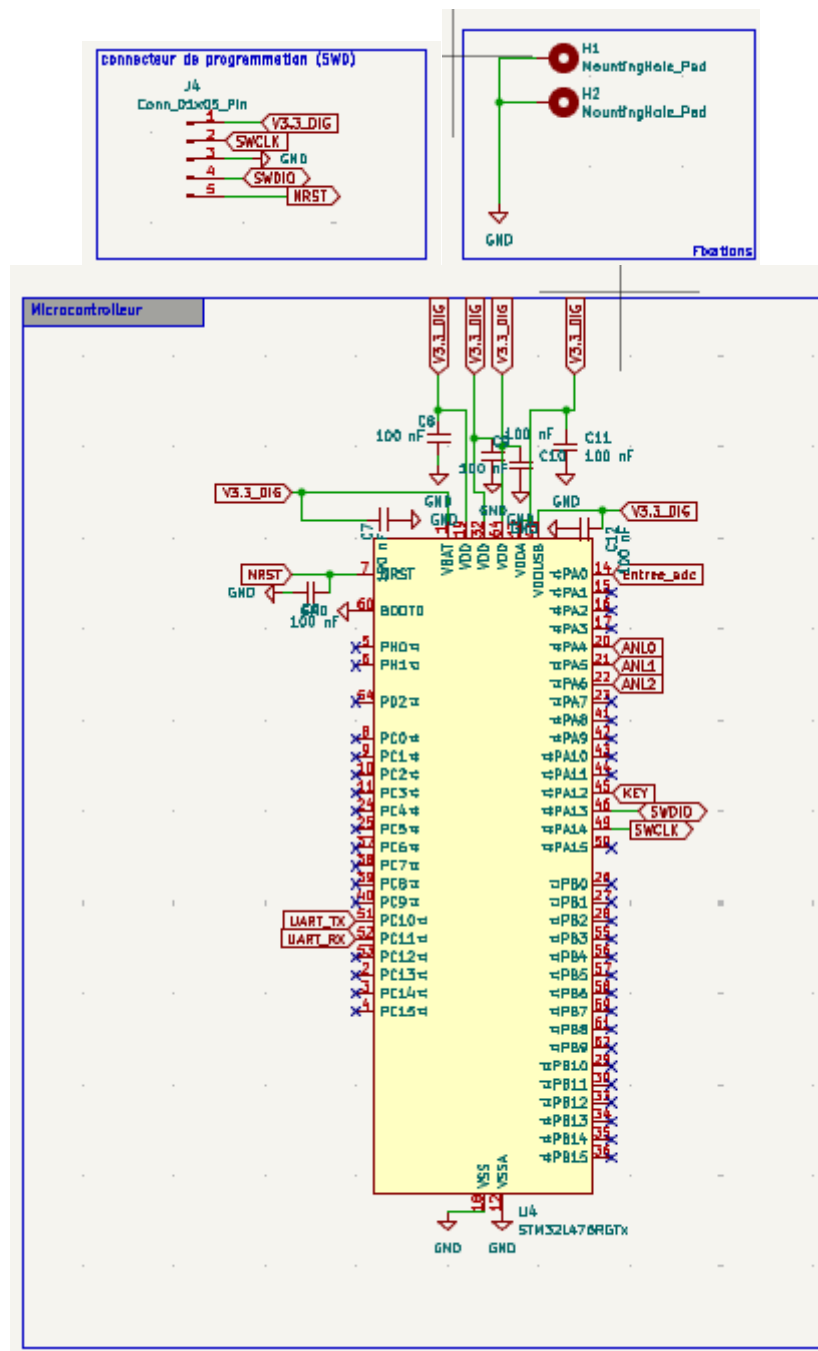
Format double couche, dimensions $\sim 24 \times 95 \text{ mm}$, coins arrondis. Organisation des couches :

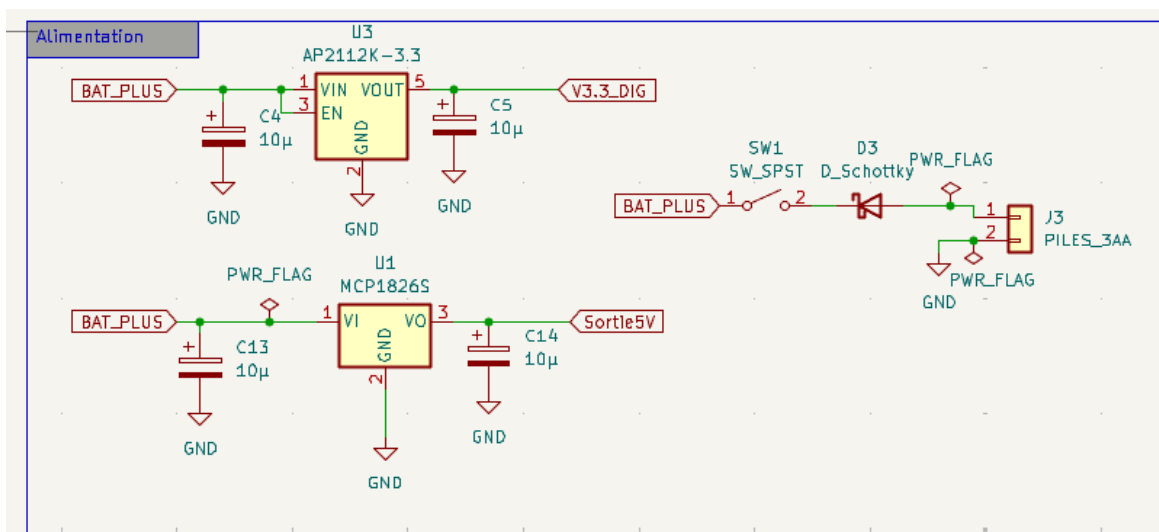
- F.Cu : signaux, pistes 0,25 mm minimum, composants CMS (STM32 LQFP-64, PGA849 QFN-16, Traco TSM0515D, LDO, condensateurs)
- B.Cu : plan de masse GND continu (copper fill) — isole les perturbations HF du Traco et garantit un retour de courant propre

Connectique intégrée : BNC vertical THT (entrée signal), module HC-05 (Bluetooth), connecteur SWD 5 broches J4 (SWCLK, SWDIO, NRST, GND, 3,3 V) pour programmation et debug sans démonter la carte, interrupteur glissière marche/arrêt, support piles 3×AA, 2 vis M3 pour fixation boîtier.

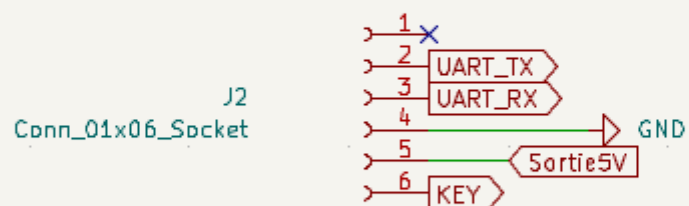
3.4.2 Règles de routage appliquées

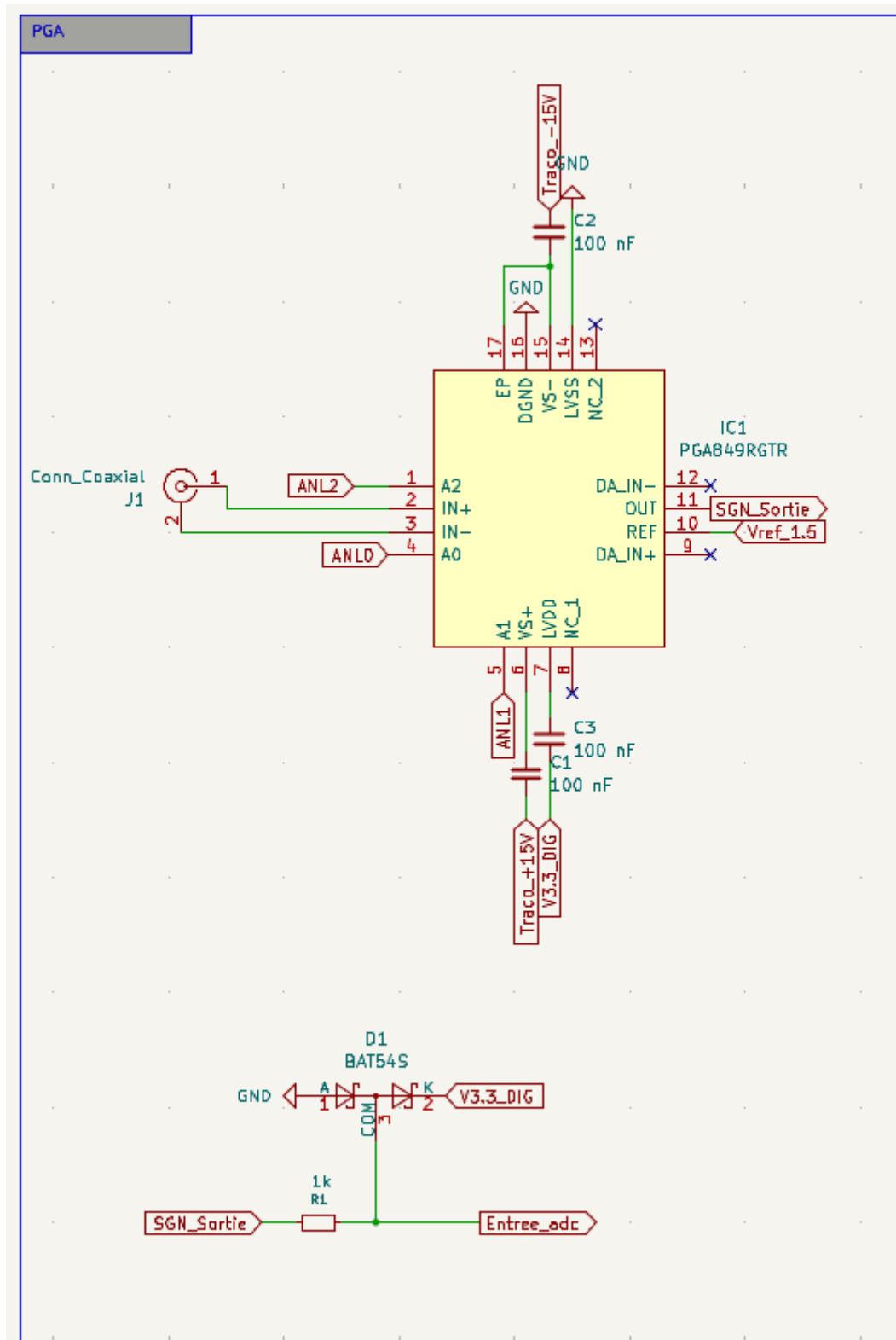
- Isolation analogique : BNC et PGA849 regroupés, distants des zones numériques (STM32, HC-05). Le plan de masse continu B.Cu sépare physiquement les domaines.
- Découplage au plus près : les 100 nF sont placés à moins de 2 mm de chaque broche d'alimentation des ICs.
- Empreintes personnalisées : PGA849 QFN-16 et TSM0515D créées manuellement dans la librairie Empreintes.pretty — les bibliothèques standard KiCad ne les incluent pas.
- DRC validé sans erreur — la carte a été envoyée en fabrication et reçue.

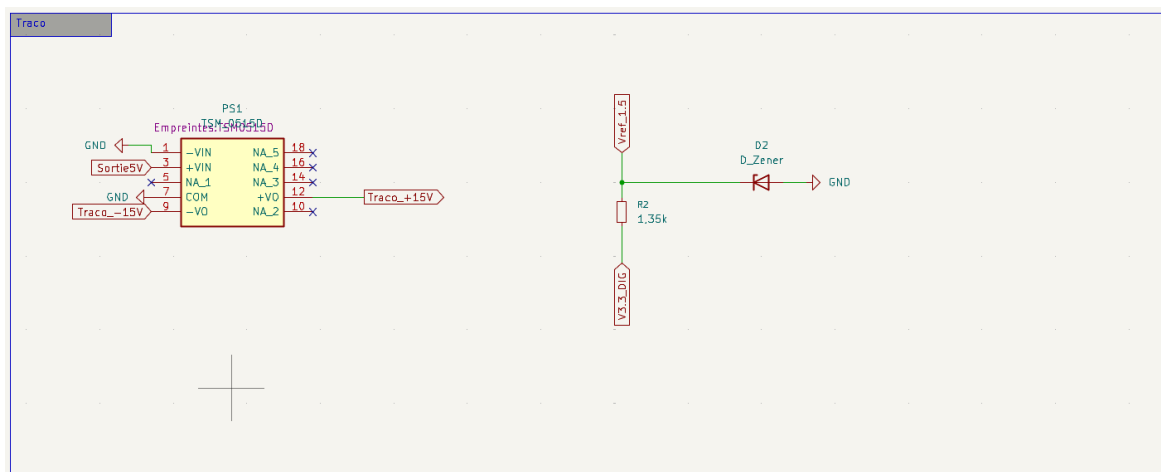


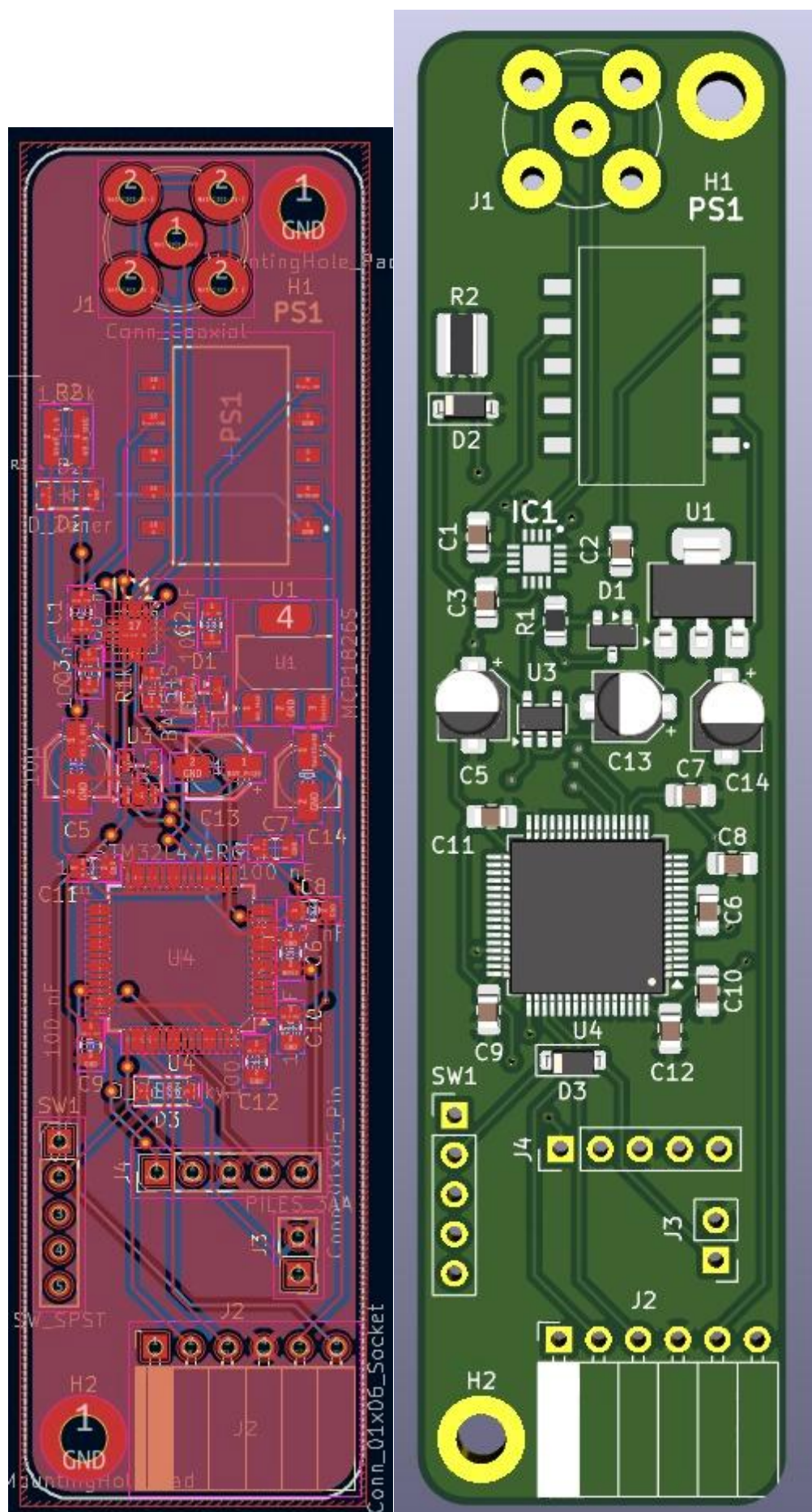


Module bluetooth HC05









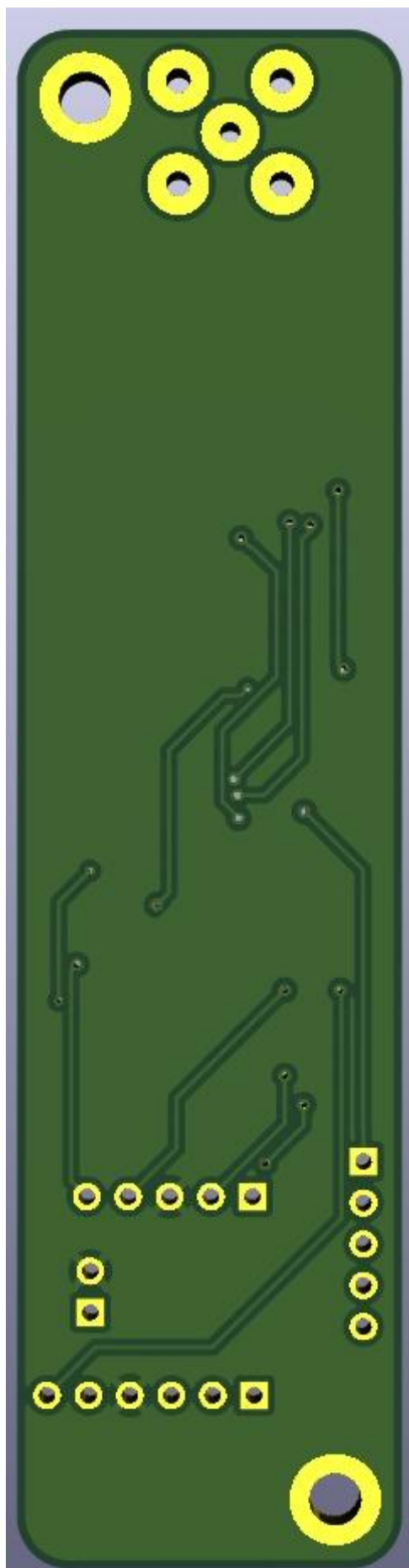


Fig. 2 — Schematic et PCB routé sous KiCad (vue F.Cu). Format ~24×95 mm, double couche. DRC validé.

4. Firmware STM32 — Architecture et Déterminisme

4.1 Vue d'ensemble

Le firmware est développé en C avec la HAL STM32 (configurée via STM32CubeMX 6.15.0). Il est organisé en modules fonctionnels indépendants, ce qui a facilité le développement en équipe et le débogage par sous-système. Les modules principaux :

Module	Rôle
main.c	Machine à états, boucle principale, orchestration
acquisition.c/.h	ADC1 + TIM2 + DMA circulaire double buffer
pga849.c/.h	Driver PGA849 — contrôle gain via GPIO PA4/5/6
protocol.c/.h	Construction/parsing trames Bluetooth (0xAA/0x55 + checksum XOR)
uart_handler.c/.h	Gestionnaire UART HC-05 — TX DMA, RX interruption, file commandes
freq_measure.c/.h	Estimation fréquence par zero-crossing (méthode logicielle)
freq_hw.c/.h	Mesure fréquence hardware via TIM3 Input Capture (PB4, AF2)

4.2 Machine à états

Le firmware suit une machine à états à trois niveaux : SYSTEM_INIT → IDLE → ACQUISITION. Cette structuration garantit que le système ne démarre jamais l'ADC avant que tous les périphériques soient correctement initialisés, et permet de gérer proprement les transitions pause/reprise commandées par Bluetooth.

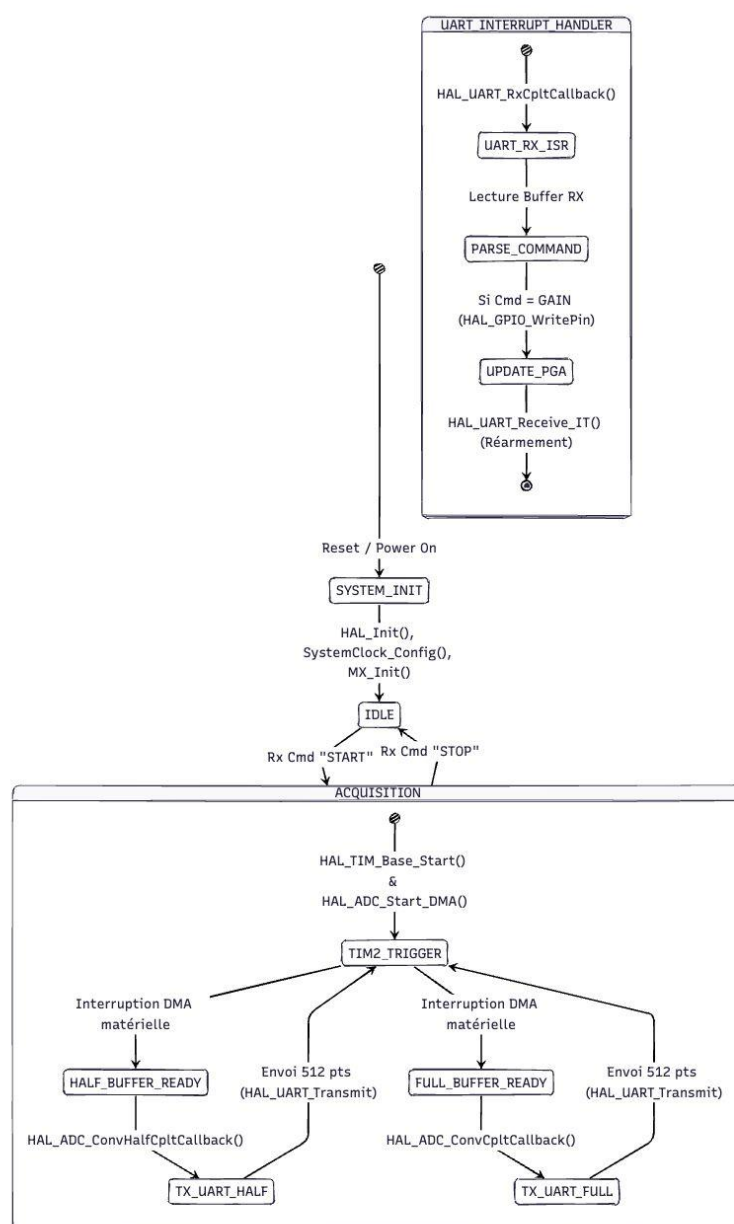


Fig. 3 — Machine à états du firmware STM32. *SYSTEM_INIT* → *IDLE* → *ACQUISITION* DMA, piloté par interruptions *TIM2*.

4.3 Acquisition ADC — DMA Circulaire Double Buffer

4.3.1 Principe et déterminisme

L'acquisition est le sous-système le plus critique en termes de déterminisme. L'objectif est d'échantillonner le signal à exactement 10 kHz, sans aucune perte d'échantillon, tout en permettant au CPU de traiter et transmettre les données en parallèle.

La solution retenue repose sur trois mécanismes combinés :

4. TIM2 génère un événement UPDATE à 10 kHz (SYSCLK=80 MHz, PSC=7, ARR=999), qui déclenche directement l'ADC via TRGO — le timing est matériel, parfaitement régulier, indépendant du CPU.
5. L'ADC dépose chaque résultat en RAM via DMA1 (mode circulaire) — aucune intervention CPU pendant les conversions.
6. Le buffer DMA de 1024 samples est divisé en deux moitiés : pendant que le CPU traite la première moitié, le DMA remplit la seconde, puis les rôles s'inversent.

Ce mécanisme de double buffering garantit deux propriétés essentielles : zéro perte d'échantillon (le flux ADC n'est jamais interrompu) et charge CPU $\approx 0\%$ pendant l'acquisition (le DMA travaille de manière totalement autonome).

DMA Circular Buffer — zéro perte de donnée

L'ADC est déclenché par TIM2 à 10 kHz. Chaque valeur est déposée en RAM par le DMA sans passer par le CPU. Le buffer de 1024 points est découpé en deux moitiés qui s'alternent :



au cycle suivant, ils s'inversent — flux ADC jamais interrompu

DMA Circular Buffer — double buffering. Les deux moitiés s'alternent : flux ADC jamais interrompu, zéro perte d'échantillon.

4.3.2 Implémentation — Callbacks HAL et synchronisation

Les deux callbacks HAL signalent la disponibilité de chaque moitié. Un flag volatile `ACQ_Status_t` est mis à jour dans l'ISR, puis consommé par la boucle principale :

```
// acquisition.c – callbacks DMA ISR (interruptions)
void HAL_ADC_ConvHalfCpltCallback(ADC_HandleTypeDef *hadc) {
    if (hadc->Instance == ADC1)
        acq_status = ACQ_HALF_READY; // samples[0..511] prêts
}

void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef *hadc) {
    if (hadc->Instance == ADC1)
        acq_status = ACQ_FULL_READY; // samples[512..1023] prêts
}

// main.c – boucle principale (poll du flag, non bloquant)
ACQ_Status_t status = ACQ_GetStatus();
if (status != ACQ_NONE) {
    const uint16_t *half = ACQ_GetReadyHalf();
    ACQ_ClearStatus(); // ← AVANT le traitement (ordre critique !)
    if (half != NULL && app_running) {
        // Calcul min / max / moyenne sur 512 samples
    }
}
```

```

        // Estimation fréquence (zero-crossing ou TIM3 IC)
        // Envoi trame Bluetooth si TX DMA libre
    }
}

```

L'ordre d'appel de `ACQ_ClearStatus()` est fondamental : il doit précéder le traitement des données. Si le flag est effacé après, la boucle peut ignorer un callback DMA qui survient pendant le traitement, et la fréquence effective d'acquisition chute de moitié. Ce bug a été découvert et corrigé lors des tests sur Nucleo (voir section 7.2).

4.4 Protocole Bluetooth — Trame et Endianness

4.4.1 Structure de la trame

Chaque demi-trame de 512 samples est encapsulée dans un paquet binaire avec checksum XOR pour la détection d'erreurs :

Champ	Taille	Valeur / Description
Header 1	1 octet	0xAA — marqueur de début
Header 2	1 octet	0x55 — marqueur de synchronisation
CMD	1 octet	0x01 = DATA, 0x02 = ACK, 0x03 = SYSINFO
LEN	2 octets	Longueur payload en big-endian
PAYLOAD	1024 octets	512 samples × 2 octets, encodés big-endian
CHECKSUM	1 octet	XOR de tous les octets depuis CMD inclus

Taille totale d'une trame DATA : 1030 octets. À 115 200 bps, la durée d'émission est d'environ 90 ms, ce qui laisse 51,2 ms de marge par rapport à la période de la demi-trame (~51,2 ms à 10 kHz / 512 samples). Le débit est donc au limit — le flag `tx_busy` dans `uart_handler.c` garantit qu'on ne lance pas un nouvel envoi DMA avant que le précédent ne soit terminé.

4.4.2 Problème d'Endianness — diagnostic et correction

Un bug non trivial a été découvert lors des premiers tests de communication : l'application Android affichait des valeurs complètement incohérentes alors que le firmware envoyait des données visiblement correctes côté STM32.

La cause : l'architecture ARM Cortex-M4 du STM32 est nativement little-endian — les octets d'un `uint16_t` sont stockés dans l'ordre LSB d'abord, MSB ensuite. Or Java (et la JVM Android) traite les types multi-octets en big-endian : MSB d'abord. Un sample de valeur 0x0C00 (3072 en décimal, ~2,4 V) était donc interprété par l'application comme 0x000C (12 en décimal, ~10 mV).

La correction a été appliquée dans `PROTO_BuildDataFrame()` : chaque sample `uint16_t` est explicitement inversé octet par octet avant d'être écrit dans le buffer de transmission :

```

// protocol.c - inversion Endianness à l'encodage
for (uint16_t i = 0; i < nb_samples; i++) {
    tx_buf[idx++] = (uint8_t)(samples[i] >> 8);    // MSB en premier (big-
    tx_buf[idx++] = (uint8_t)(samples[i] << 8);    // endian)
}

```

```
tx_buf[idx++] = (uint8_t)(samples[i] & 0xFF); // LSB ensuite
// → Java lit naturellement : (buf[0] << 8) | buf[1] = valeur correcte
}
```

Ce bug illustre bien une difficulté classique des systèmes embarqués hétérogènes : l'interopérabilité entre architectures little-endian et big-endian nécessite une gestion explicite à chaque frontière de sérialisation.

4.4.3 Commandes Android → STM32

Les commandes depuis le téléphone vers le firmware sont encodées sur un seul octet ASCII, ce qui simplifie le parseur et évite de gérer une machine à états de parsing côté STM32 :

```
// protocol.h – commandes RX (un seul octet ASCII)
#define PROTO_RX_GAIN_MIN '0' // codes '0'..'7' = changement gain PGA
#define PROTO_RX_GAIN_MAX '7'
#define PROTO_RX_SYSINFO 'S' // demande infos système (Fs, gain, Vref)
#define PROTO_RX_PAUSE 'P' // pause acquisition (TIM2 stop)
#define PROTO_RX_RESUME 'R' // reprise acquisition (TIM2 start)
```

4.5 Driver PGA849 — Contrôle gain

Le driver est volontairement minimal. Il écrit 3 bits GPIO en sortie push-pull et maintient un lookup table statique pour la conversion code → multiplicateur réel :

```
// pga849.c – table de correspondance et contrôle GPIO
static const uint16_t lut[8] = { 1, 2, 5, 10, 20, 50, 100, 200 };

void PGA_SetGain(PGA_Gain_t gain) {
    gain &= 0x07;
    HAL_GPIO_WritePin(PGA_A0_GPIO_Port, PGA_A0_Pin,
                      (gain & 0x01) ? GPIO_PIN_SET : GPIO_PIN_RESET);
    HAL_GPIO_WritePin(PGA_A1_GPIO_Port, PGA_A1_Pin,
                      (gain & 0x02) ? GPIO_PIN_SET : GPIO_PIN_RESET);
    HAL_GPIO_WritePin(PGA_A2_GPIO_Port, PGA_A2_Pin,
                      (gain & 0x04) ? GPIO_PIN_SET : GPIO_PIN_RESET);
    current_gain = gain;
}
```

Le changement de gain est sécurisé : ACQ_Suspend() stoppe TIM2, le nouveau gain est appliqué, un délai de settling d'1 ms est respecté (selon la datasheet PGA849), puis ACQ_Resume() relance TIM2. Un ACK est envoyé à l'application pour confirmation.

4.6 Estimation de fréquence

4.6.1 Méthode logicielle — Zero-crossing

La fréquence est estimée par comptage de fronts montants dans la demi-trame de 512 samples. L'algorithme calcule la moyenne du buffer (référence DC locale), compte les passages de en-dessous vers au-dessus, et en déduit :

$$f = F_s \times N_{\text{crossings}} / N_{\text{samples}} = 10000 \times N / 512$$

Cette méthode est suffisante pour des signaux périodiques propres entre ~ 20 Hz et 4 kHz. Elle dégrade au-delà (approche de Nyquist) et pour les signaux très bruités.

4.6.2 Méthode hardware — TIM3 Input Capture

Une méthode plus précise est disponible via `freq_hw.c` : le TIM3 est configuré en Input Capture sur PB4 (AF2, 1 MHz de résolution avec PSC=79). La période entre deux fronts montants consécutifs est mesurée avec une résolution de 1 μ s. Cette méthode est prioritaire si elle retourne une valeur non nulle — elle nécessite un signal numérique mis en forme sur PB4, ce qui n'est pas implémenté dans le PCB V1 mais est prévu pour la V2.

5. Application Android Java

5.1 Présentation et rôle dans le projet

L'application Android est le composant le plus abouti du projet. Développée en Java natif Android (AppCompatActivity), elle offre une interface d'oscilloscope complète : affichage temps réel, trigger, FFT, curseurs, mesures automatiques. Elle intègre également un mode simulation qui génère des signaux synthétiques en interne, ce qui a servi de banc de test logiciel pendant tout le développement, en l'absence de hardware complet.

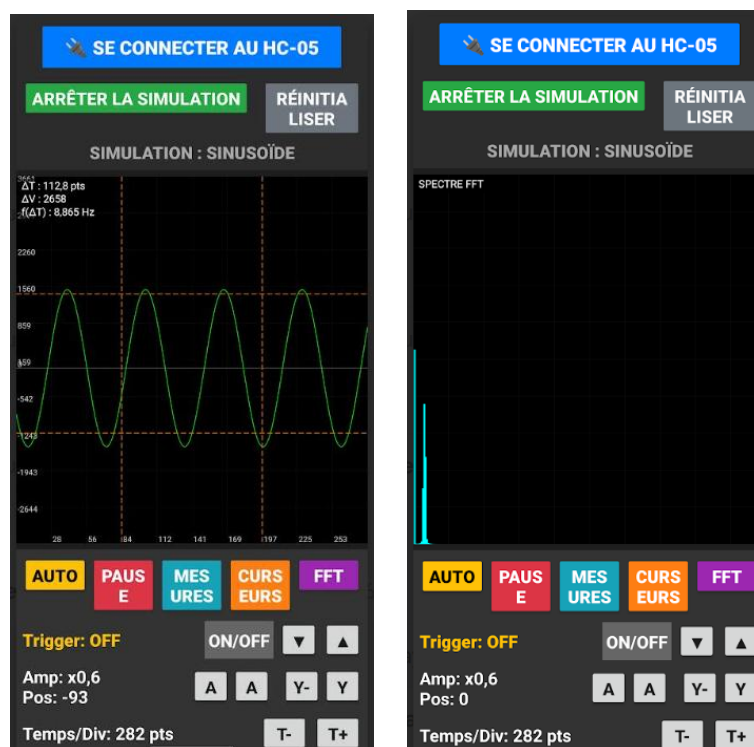


Fig. 5 — Application Android (mode simulation). Affichage temps réel, contrôles gain/trigger/FFT.

5.2 Architecture — Diagramme de classes

L'application repose sur deux classes principales qui communiquent via l'interface `OnValuesChangeListener` :

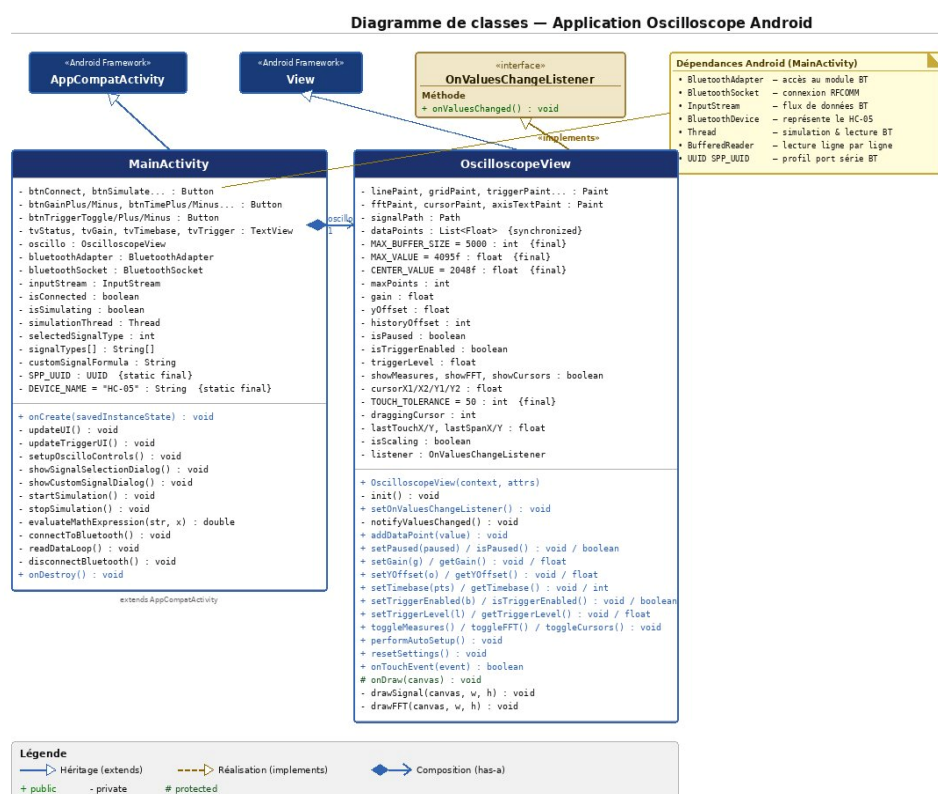


Fig. 4 — Diagramme de classes Java. MainActivity ↔ OscilloscopeView via OnValuesChangeListener.

5.2.1 MainActivity

MainActivity étend AppCompatActivity et orchestre l'ensemble de l'application. Ses responsabilités :

- Connexion Bluetooth RFCOMM au HC-05 (UUID SPP : 00001101-0000-1000-8000-00805F9B34FB)
- Lecture continue de l'InputStream Bluetooth dans un Thread dédié (readDataLoop)
- Mode simulation : un simulationThread génère des signaux synthétiques à fréquence et amplitude configurables
- Contrôles UI : boutons Connect/Simulate, réglages gain/timebase/trigger, toggles FFT/curseurs/mesures
- Gestion des signaux personnalisés : evaluateMathExpression() permet d'entrer une formule mathématique ($\sin(x)+0.5*\cos(3*x)$, etc.)

5.2.2 OscilloscopeView

OscilloscopeView étend View et réalise l'intégralité du rendu graphique et du traitement du signal. C'est le composant le plus complexe de l'application. Attributs principaux :

- dataPoints (List<Float>, synchronisé) : buffer circulaire des données, MAX_BUFFER_SIZE = 5000 points
- gain, yOffset : mise à l'échelle verticale et décalage

- timebase : nombre de points affichés horizontalement (zoom temporel)
- triggerLevel, isTriggerEnabled : paramètres du trigger
- showFFT, showMeasures, showCursors : toggles des affichages supplémentaires
- cursorX1/X2/Y1/Y2 : positions des 4 curseurs déplaçables

5.3 Fonctionnalités

5.3.1 Affichage et trigger

La méthode `onDraw(Canvas)` trace le signal via `Canvas.drawPath()`. Le trigger est implémenté par recherche du premier front montant dépassant `triggerLevel` — même principe que le firmware, ce qui garantit une cohérence entre les deux représentations. L'utilisateur règle le seuil depuis l'interface (+/- par pas de 100 unités sur l'échelle 0–4095).

5.3.2 Zoom et navigation tactile

`onTouchEvent()` gère deux gestes : pinch à deux doigts (zoom horizontal timebase et vertical gain, via distance inter-doigts entre `ACTION_MOVE`) et glissement à un doigt (navigation historique `historyOffset` et décalage vertical `yOffset`). La détection du passage pinch ↔ glissement se fait sur `isScaling`.

5.3.3 Analyse FFT

L'application intègre une FFT de 512 points (algorithme de Cooley-Tukey Decimation-In-Time, implémenté en Java). Une fenêtre de Hann est appliquée avant le calcul pour réduire les lobes secondaires. Le spectre est affiché en barres verticales dans la partie basse de l'écran, en temps réel. Ce module a été validé en simulation : un sinus à 1 kHz ($F_s = 10$ kHz simulé) produit bien un pic spectral unique à la bonne fréquence.

5.3.4 Mesures automatiques — `performAutoSetup()`

La méthode `performAutoSetup()` analyse les 2000 derniers points pour calculer $V_{\max}$, $V_{\min}$, V_{moy} , période par zero-crossing, puis ajuste automatiquement gain, `yOffset` et `triggerLevel` pour centrer le signal à l'écran. C'est l'équivalent du bouton « Auto » d'un oscilloscope classique. En mode mesures actif, l'application affiche en continu $V_{\max}$, $V_{\min}$, $V_{\text{cc}} = V_{\max} - V_{\min}$ et la fréquence estimée.

5.3.5 Curseurs ΔT / ΔV

4 curseurs déplaçables par toucher (2 en X, 2 en Y) affichent en temps réel : $\Delta T = |X_2 - X_1| / F_s$, $\Delta V = |Y_2 - Y_1|$ mis à l'échelle, et $f = 1/\Delta T$. La sélection du curseur à déplacer se fait par proximité (`TOUCH_TOLERANCE = 50` px).

5.3.6 Mode simulation

Le `simulationThread` génère des signaux synthétiques (sinus, carré, triangle, ou formule personnalisée) à la même cadence que les données réelles attendues. Ce mode a été utilisé pour valider toutes les fonctionnalités de l'interface de manière indépendante du hardware : trigger, FFT, curseurs, mesures automatiques, zoom tactile. Il constitue un banc de test logiciel complet et a validé le bon fonctionnement de l'ensemble de la chaîne de traitement côté application.

```
// MainActivity.java - exemple : lecture Bluetooth en Thread
private void readDataLoop() {
    try {
        BufferedReader reader = new BufferedReader(
            new InputStreamReader(inputStream));
```



```

        while (isConnected) {
            String line = reader.readLine();
            if (line != null) {
                float value = Float.parseFloat(line.trim());
                oscillo.addDataPoint(value); // → OscilloscopeView
            }
        }
    } catch (IOException e) {
        disconnectBluetooth();
    }
}

```

6. Validation du Système

6.1 Synthèse de ce qui a été validé

Composant	Type de validation	Résultat
Firmware — DMA/ADC (Nucleo)	Hardware partiel (sans PGA)	OK — buffer circulaire fonctionnel
Firmware — UART Bluetooth	Hardware (HC-05 + terminal)	OK — trames 0xAA/0x55 reçues et vérifiées
Firmware — Endianness	Test fonctionnel sur Nucleo	Bug détecté et corrigé
Protocole — Checksum XOR	Test sur 100 trames consécutives	OK — 0 erreur à 1 m
App Android — Affichage	Mode simulation (tous types)	OK — rendu graphique validé
App Android — FFT 512 pts	Simulation sinus 1 kHz	OK — pic spectral à la bonne fréquence
App Android — Trigger	Simulation sinus et carré	OK — signal stable à l'écran
App Android — Curseurs	Mode simulation	OK — $\Delta T/\Delta V$ calculés correctement
Chaîne complète BNC→App	Non réalisé (PGA849 manquant)	Non validé
PCB assemblé complet	Non réalisé (composants manquants)	Non validé

6.2 Tests firmware sur Nucleo

Le firmware a été testé sur la carte Nucleo-L476RG avec un signal analogique injecté directement sur PA0 depuis un générateur de fonctions, en court-circuitant le PGA849. Les tests ont confirmé le bon fonctionnement du DMA circulaire (alternance HalfCplt/Cplt vérifiée par GPIO toggle sur oscilloscope), la cohérence des valeurs ADC avec le signal injecté (debug UART sur USART2/ST-Link VCP), et la correction du bug de synchronisation DMA après sa détection.

6.3 Tests Bluetooth

La liaison HC-05 ↔ application Android a été testée à 1 m, en envoyant des trames DATA simulées depuis le firmware (valeurs générées sans ADC réel). Résultats : 0 erreur checksum sur 100 trames, latence ~90 ms par trame cohérente avec le calcul théorique (1030 octets à 115 200 bps), parsing correct côté application.

6.4 Limites de validation

Trois aspects importants n'ont pas pu être validés expérimentalement :

- Précision de mesure de tension avec PGA849 sur toute la plage de gain ($\times 1$ à $\times 200$)
- Bande passante réelle du système (objectif 50 kHz, non mesurée)
- Autonomie réelle sur batterie (objectif 6 h, non mesurée)

Ces validations sont directement planifiées pour la suite du projet, dès réception et soudure des composants manquants.

7. Difficultés Rencontrées

7.1 Retard critique de livraison Mouser

La difficulté principale du projet a été logistique. Le PGA849, commandé sur Mouser, a été livré avec plus de 3 semaines de retard sur la date annoncée. Sans ce composant, l'assemblage de la chaîne d'acquisition complète était impossible. D'autres CMS passifs de précision ont subi des délais similaires.

Cette contrainte nous a forcés à revoir l'organisation du travail : concentration sur le firmware et l'application Android, qui pouvaient progresser indépendamment. Le mode simulation de l'application a alors pris une importance centrale comme outil de validation. En retrospective, commander les composants critiques dès la première semaine du projet avec une marge de sécurité importante aurait évité cette situation.

7.2 Bug de synchronisation DMA — ACQ_ClearStatus

Un bug subtil a affecté le firmware pendant plusieurs séances. L'appel à `ACQ_ClearStatus()` était placé après le traitement des données au lieu de le précéder. En conséquence, un callback DMA pouvait survenir pendant le traitement, le flag était alors écrasé par l'ISR, puis effacé par la boucle principale — et le prochain event DMA était ignoré. L'acquisition fonctionnait mais à la moitié de la fréquence attendue, sans message d'erreur visible.

Le diagnostic a nécessité l'utilisation d'un GPIO en mode toggle pour observer les callbacks ISR sur oscilloscope. La correction (déplacer `ACQ_ClearStatus()` avant le traitement) était triviale une fois le problème compris.

7.3 Problème d'Endianness (Little-Endian STM32 vs Big-Endian Java)

Décrit en détail en section 4.4.2. Ce bug a provoqué des valeurs absurdes côté application lors des premiers tests de communication. Le diagnostic a pris du temps car les données semblaient correctes côté STM32 (vérifiées par UART debug) mais aberrantes côté Android. La prise de conscience de la différence d'endianness entre les deux architectures a orienté vers la solution.

7.4 Empreintes KiCad personnalisées

Les empreintes du PGA849 (QFN-16, pas 0,5 mm) et du TSM0515D n'existaient pas dans les bibliothèques standard KiCad. Les modèles trouvés en ligne avaient souvent des dimensions légèrement incorrectes. Elles ont finalement été recrées entièrement à partir des datasheets, en vérifiant chaque cote — un travail fastidieux mais nécessaire pour garantir l'assemblage.

7.5 Contraintes de temps et d'organisation

Quatre membres d'équipe avec des emplois du temps différents, des cours en parallèle, des partiels — la coordination n'a pas toujours été simple. La décomposition en modules indépendants (firmware, application, PCB) a aidé à paralléliser le travail, mais les phases d'intégration ont révélé des incompatibilités (endianness, timing) qui auraient pu être anticipées par plus de communication inter-équipe.

8. Améliorations et Perspectives

8.1 Court terme — Finalisation V1

- Terminer l'assemblage PCB dès réception des composants et valider la chaîne complète.
- Caractériser précision de tension, bande passante et autonomie sur batterie.
- Valider le mode simulation vs acquisition réelle pour calibrer les paramètres d'affichage.

8.2 Hardware — V2

- Filtre anti-repliement actif : Sallen-Key 2ème ordre, $F_c = 4,5$ kHz, avant l'ADC (évite le repliement à Nyquist 5 kHz).
- Seconde voie d'acquisition : ADC2 + second PGA849 pour le mode dual conforme au cahier des charges (mesure de déphasage).
- Mise en forme signal pour TIM3 IC : comparateur à hystérésis LM393 sur PB4 — active la mesure hardware de fréquence (résolution 1 μ s vs zero-crossing logiciel).
- LiPo rechargeable + circuit TP4056 : remplacement des piles AA pour un usage plus pratique et rechargeable.
- Augmentation de F_s à 100–200 kHz : oversampling ADC STM32, DMA plus rapide, pour atteindre les 50 kHz de bande passante spécifiés.

8.3 Logiciel — V2

- Mode XY (Lissajous) sur l'application quand la 2ème voie sera disponible.
- Export des captures en CSV ou image depuis l'application Android.
- Protocole Bluetooth robustifié : CRC-16 + mécanisme d'ACK/retransmission.
- Auto-calibration du gain dans le firmware (ajustement automatique du PGA en fonction du niveau du signal).

9. Conclusion

Ce projet de semestre 6 a été une expérience complète et représentative des réalités du développement d'un système embarqué : bonnes surprises côté logiciel, aléas côté approvisionnement. Le bilan est nuancé, mais loin d'être négatif.

Sur la partie hardware, la conception est aboutie. Le schéma électronique, l'architecture d'alimentation (cascade BU33SD5WG 68 dB PSRR + TSM0515D, protection BAT54S), le PCB double couche $\sim 24 \times 95$ mm avec isolation analogique/numérique — tout cela représente un travail de conception rigoureux et justifié. Le retard Mouser sur le PGA849 a empêché la validation expérimentale complète, mais la carte est prête à être assemblée et mise en service. Le système est ready-to-plug.

Sur la partie logicielle, le projet est pleinement fonctionnel. Le firmware STM32 (DMA circulaire double buffer, machine à états, protocole Bluetooth avec correction d'endianness) a été testé et validé sur Nucleo. L'application Android Java — affichage temps réel, FFT 512 points fenêtrée, trigger, curseurs $\Delta T/\Delta V$, mesures automatiques, mode simulation — est opérationnelle et a servi de banc de test logiciel pendant tout le développement.

Du point de vue des compétences, ce projet a couvert un spectre remarquablement large : conception schématique, PCB, alimentation analogique, firmware bas niveau (timers, DMA, UART, interruptions), protocoles de communication, et développement mobile Android. C'est précisément l'articulation hardware/software qui définit le métier d'ingénieur en systèmes embarqués, et c'est ce qu'on a appris à maîtriser ici.

La leçon principale à retenir pour un prochain projet : dans tout projet hardware, les délais d'approvisionnement doivent être traités comme un risque de premier ordre. Commander les composants critiques dès la première semaine, avec des marges conservatrices, aurait changé le résultat final. C'est une erreur qu'on ne refera pas.

Annexes

Annexe A — Bons de commande composants

Fournisseur	Composant / Désignation	Qté	Prix unitaire
Farnell	Commutateur glissière (2435104)	10	0,50 €
Farnell	Diode Zener (1459115)	5	0,15 €
Farnell	Régulateur MCP1826S (1578426)	1	0,79 €
Farnell	Régulateur AP2112K-3.3 (3257429RL)	5	0,22 €
Farnell	Diode Schottky BAT54S	5	0,10 €
Farnell	Condensateur électrolytique 10 μ F (2466203)	5	0,20 €
Farnell	Convertisseur DC/DC TSM0515D (1205075)	1	9,60 €
Farnell	Support batterie BH4AAW (4305628)	1	3,80 €
Mouser	PGA849RGTR (amplificateur à gain prog.)	1	8,60 €
Mouser	Connecteur coaxial BNC — TE 1478204	1	3,88 €
Robot-maker	Module Bluetooth HC-05	1	8,40 €

Annexe B — Configuration STM32CubeMX

Périphérique	Configuration
MCU	STM32L476RGT3 — LQFP-64 — Board NUCLEO-L476RG
SYSCLK	80 MHz (HSI 16 MHz → PLL $\times 10 \div 2$ via PLLR)

ADC1	PA0 (IN5), 12 bits, single-ended, Trig = TIM2_TRGO ↑, DMA circulaire
TIM2	PSC=7, ARR=999, CounterMode=UP, TRGO=UPDATE → 10 kHz
TIM3	Input Capture CH1 (PB4, AF2), PSC=79 → résolution 1 µs
USART1	PA9=TX, PA10=RX, 115 200 bps, TX en DMA (Ch4), RX en IT
USART2	Debug ST-Link VCP, 115 200 bps
DMA1 Ch1	ADC1 → mémoire, PERIPH_TO_MEM, HALFWORD, circulaire, priorité 1
DMA1 Ch4	USART1 TX, MEM_TO_PERIPH, BYTE, normal, priorité 2
GPIO PA4/5/6	Output PP — contrôle gain PGA849 (A0/A1/A2)
GPIO PC13	Input EXTI falling — bouton B1, anti-rebond 200 ms software

Annexe C — Arborescence du projet firmware

Oscillo V1/	
Core/	
Inc/	
acquisition.h	DMA circulaire double buffer
freq_hw.h	TIM3 Input Capture fréquence
freq_measure.h	Zero-crossing fréquence
main.h	Defines globaux, aliases GPIO
pga849.h	Driver PGA849 gain programmable
protocol.h	Protocole 0xAA/0x55, endianness
uart_handler.h	UART HC-05, file commandes RX
Src/	
acquisition.c	ADC/DMA, callbacks HAL ISR
freq_hw.c	TIM3 IC, mesure période
freq_measure.c	Zero-crossing, estimation f
main.c	Machine à états, boucle principale
pga849.c	GPIO PA4/5/6, LUT gain
protocol.c	Build/parse trames, swap endianness
uart_handler.c	TX DMA, RX IT, file FIFO commandes
Drivers/	
STM32L4xx_HAL_Driver/	HAL ST (non modifiée)
Oscillo V1.ioc	Configuration STM32CubeMX 6.15.0

Annexe D — Classes Java application Android

Classe / Interface	Rôle
MainActivity	Activité principale — Bluetooth RFCOMM, simulation, contrôles UI
OscilloscopeView	View personnalisée — rendu signal, FFT, trigger, curseurs, zoom tactile
OnValuesChangeListener	Interface callback OscilloscopeView → MainActivity (gain, timebase...)
BluetoothAdapter	Accès Bluetooth Android (framework)

BluetoothSocket	Connexion RFCOMM au HC-05 via UUID SPP
Thread readDataLoop	Lecture continue InputStream Bluetooth, parsing valeurs flottantes
Thread simulationThread	Génération signaux synthétiques (sin, carré, triangle, formule custom)